



PERTEMUAN X

MODEL DEPLOYMENT (FLASK)

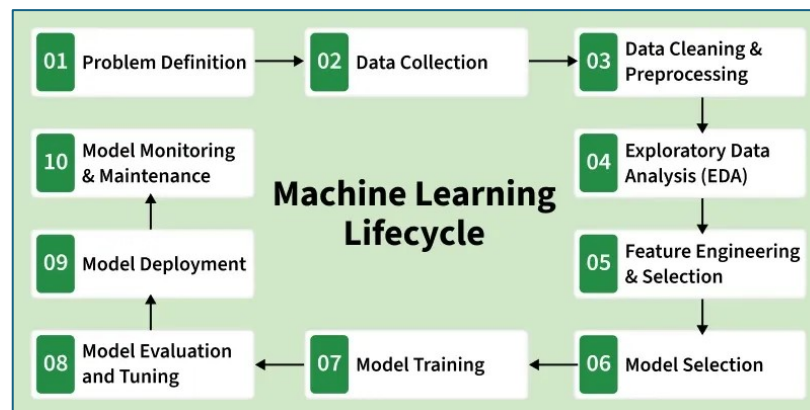
10.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar model deployment dalam siklus hidup data mining
- B. Mahasiswa mampu melakukan serialization (penyimpanan) dan deserialization (pemuatan) model machine learning menggunakan pustaka pickle atau joblib
- C. Mahasiswa dapat membangun layanan web (Web Service) sederhana menggunakan framework Flask.
- D. Mahasiswa dapat mengintegrasikan model machine learning ke dalam API sehingga dapat menerima input data baru dan memberikan hasil prediksi.

10.2. DASAR TEORI

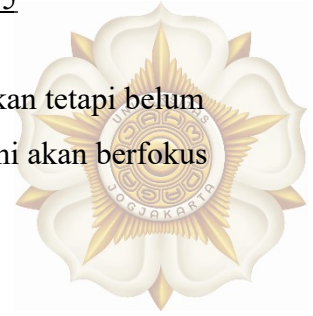
- **Tahapan Pengembangan Machine Learning**

Pengembangan model machine learning bukanlah proses satu kali jalan, melainkan sebuah siklus (cycle) yang berkelanjutan. Secara umum, tahapan ini, yang sering disebut sebagai *ML Pipeline*. Secara umum dapat terlihat seperti pada Gambar 0.1.



Gambar 0.1 Machine Learning Lifecycle (Sumber: www.geeksforgeeks.org)

Pada pertemuan sebelumnya kita telah melakukan berbagai tahapan pengembangan mulai dari *Problem Definition* hingga *Model Evaluation and Tuning*. Pada tahapan



sebelumnya kita telah mendapatkan sebuah model yang bisa digunakan tetapi belum bisa dimanfaatkan oleh orang lain. Oleh karena itu, pertemuan kali ini akan berfokus pada tahap kesembilan yaitu Model Deployment.

- **Model Deployment**

Dalam data mining, membuat model yang akurat hanyalah sebagian dari pekerjaan.

Agar model tersebut berguna, model harus di-deploy atau diterapkan ke dalam produksi sehingga dapat digunakan oleh pengguna akhir atau sistem lain. Deployment mengubah model statis menjadi layanan aktif.

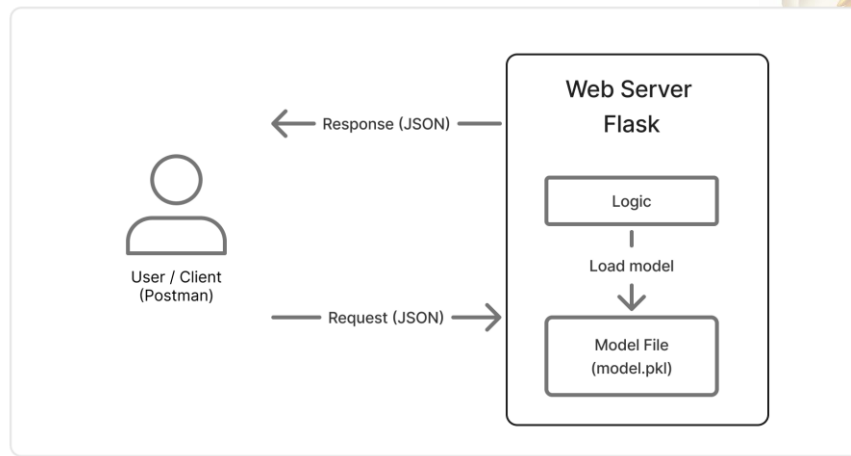
Skema deployment model bermacam-macam. Terdapat skema deployment model secara tertanam di dalam aplikasi (embedded), skema client-server, dan lain-lain. Dari banyaknya pilihan skema deployment yang dapat dilakukan, modul ini akan menggunakan skema client-server. Penggunaan aplikasi pada skema ini akan melalui perantara RESTful API.

Skema alur data pada client-server pada :

1. Flask bertindak sebagai Web Service. Ia menyediakan URL (alamat) yang bisa diakses. Flask menjembatani komunikasi antara dunia luar (internet/user) dengan logika internal Python dan Model Machine Learning. Tanpa Flask, model kita hanya bisa dijalankan di dalam komputer lokal pengembangnya saja.
2. API Endpoint: Data diterima oleh "pintu gerbang" server yang kita buat dengan Flask (misalnya di alamat /predict).
3. Web Server (Flask): Aplikasi Flask menerima data tersebut, lalu memproses format datanya agar sesuai dengan input yang dibutuhkan model.
4. Model Loading: Flask memanggil file model (.pkl) yang sudah dilatih sebelumnya. Model ini bertindak sebagai "otak" yang melakukan perhitungan matematika untuk menghasilkan prediksi.
5. Response: Hasil prediksi dikirimkan kembali oleh Flask kepada Client dalam format JSON (JavaScript Object Notation), sehingga mudah dibaca oleh manusia maupun program lain.



Penjelasan tersebut dapat diilustrasikan seperti pada Gambar 0.2.



Gambar 0.2 Skema alur data client-server

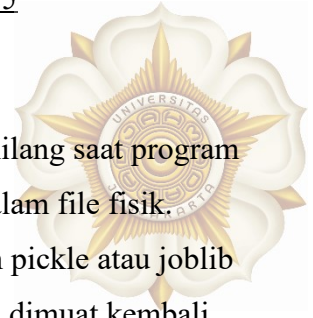
Flask bertindak sebagai Web Service. Ia menyediakan URL (alamat) yang bisa diakses. Flask menjembatani komunikasi antara dunia luar (internet/user) dengan logika internal Python dan Model Machine Learning. Tanpa Flask, model kita hanya bisa dijalankan di dalam komputer lokal pengembangnya saja.

- **Flask Framework**

Flask adalah micro-web framework yang ditulis dalam bahasa Python. Disebut "micro" karena tidak memerlukan alat atau pustaka tertentu (seperti ORM) secara bawaan. Flask sangat populer untuk membuat API (Application Programming Interface) bagi model Machine Learning karena ringan, cepat, dan mudah dipelajari. Selain Flask ada berbagai framework lain yang dapat digunakan seperti: Django, Fastapi, Tornado, ataupun Pyramid.



Gambar 0.3 Logo Flask



- **Model Serialization**

Model machine learning yang telah dilatih di memori (RAM) akan hilang saat program ditutup. Untuk itu, kita perlu menyimpan objek model tersebut ke dalam file fisik.

Proses ini disebut Serialization. Di Python, kita sering menggunakan pickle atau joblib (dari Scikit-Learn) untuk menyimpan model ke format .pkl agar bisa dimuat kembali (deserialization) oleh server Flask tanpa perlu melatih ulang model setiap kali ada permintaan.

10.3. ALAT DAN BAHAN

- **Perangkat Keras**

- Komputer/Laptop

- **Perangkat Lunak**

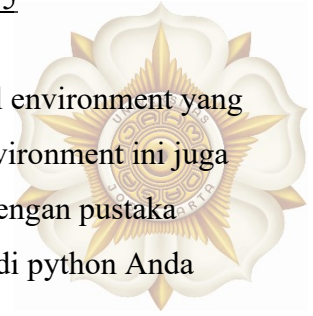
- Python3
- Text Editor / IDE. Catatan: Tidak disarankan menggunakan Jupyter Notebook untuk menjalankan server Flask.
- Postman (Opsional, untuk testing API)

- **Pustaka Python**

- tensorflow
- flask
- scikit-learn
- pandas
- joblib atau pickle

10.4. LANGKAH PERCOBAAN

Pada pembuatan modul praktikum, environment yang digunakan adalah sistem operasi Linux Ubuntu di dalam WSL, Python 3.12, dan memiliki akses GPU. Oleh karena ini beberapa perintah dapat disesuaikan dengan target environment deployment masing-masing.



Dalam membuat proyek python alangkah baiknya menggunakan virtual environment yang sesuai dengan environment produksi yang ada di server. Pembuatan environment ini juga dapat membantu dalam manajemen pustaka agar tidak terjadi konflik dengan pustaka global lainnya. Untuk mempelajari lebih lanjut mengenai environment di python Anda dapat membaca referensi berikut:

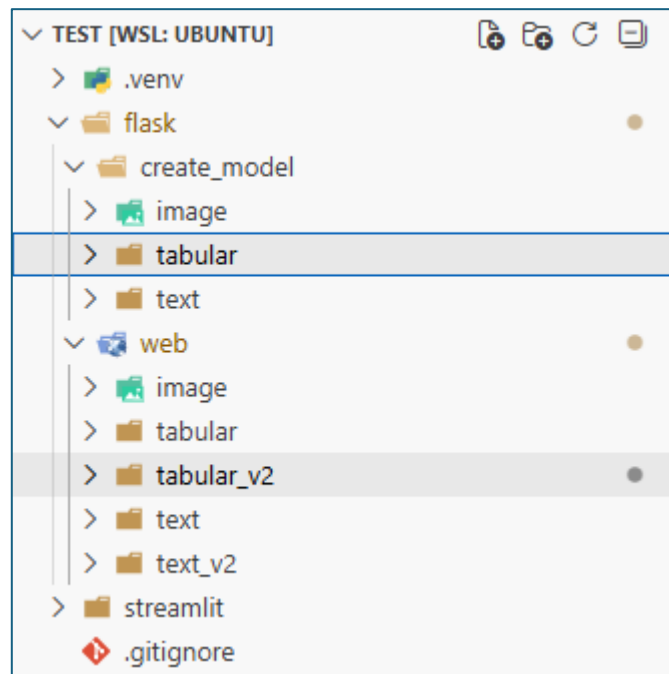
- (vanilla) <https://packaging.python.org/en/latest/guides/installing-using-pip-and-virtual-environments/>
- (UV Astral) <https://docs.astral.sh/uv/>
- (Miniconda) <https://www.anaconda.com/docs/getting-started/miniconda/main>

File dataset yang diperlukan dalam praktikum ini dapat Anda unduh pada link berikut:

<https://drive.google.com/file/d/1R5vXII015dJ0s3HYbZMvvBdcM3E6VVh2/view?usp=sharing>

Praktikum ini akan membuat aplikasi prediksi klasifikasi penyakit hati. Silakan salin folder berikut ke direktori proyek kalian:

- flask/create_model/tabular
- flask/web/tabular_v2



Gambar 0.1 Folder yang perlu disalin



Setelah menyalin folder tersebut, lakukan langkah-langkah berikut:

1. Unduh pustaka

Unduh pustaka flask, numpy, scikit-learn, pandas, dan juga tensorflow. Pada perintah berikut, pustaka tensorflow yang dipilih adalah tensorflow yang memiliki layer ke hardware GPU komputer. Sebelum Anda menyalin kode dibawah ini dan menjalankannya secara langsung, pastikan Anda telah membaca instalasi tensorflow yang sesuai dengan environment Anda di link berikut:

<https://www.tensorflow.org/install/pip#windows-wsl2>

Berikut kode instalasi PIP jika menggunakan WSL dan memiliki GPU:

```
pip install flask numpy scikit-learn pandas "tensorflow[and-gpu]" joblib
```

2. Pembuatan model dan penyimpanan model

Seperti pada praktikum sebelumnya, dalam membuat model diperlukan data preprocessing, feature engineering, dan lain-lain. Pembuatan model kali ini akan menggunakan model vanilla Decision Tree untuk klasifikasi. Cuplikan kode dibawah ini digunakan untuk menyiapkan data ke model Decision Tree.

```
1 #read the dataset
2 pd.options.display.max_columns=None
3 df_ori = pd.read_csv('hcvdat0.csv')
4 df_ori = df_ori.dropna()
5 #get the X and y
6 df_X = df_ori.drop(['Unnamed: 0', 'Category'],axis=1)
7 df_y = df_ori['Category']
8 #relabelling the class
9 df_y=df_y.replace('0=Blood Donor',0)
10 df_y=df_y.replace('0s=suspect Blood Donor',0)
11 df_y=df_y.replace('1=Hepatitis',1)
12 df_y=df_y.replace('2=Fibrosis',2)
13 df_y=df_y.replace('3=Cirrhosis',3)
14
15 #get the numerical attributes
16 df_X_num = df_X.drop(['Sex'], axis=1)
17 #convert single categorical column into numeric, label encoding
18 le = preprocessing.LabelEncoder()
19 df_X['Sex'] = le.fit_transform(df_X['Sex'])
20
21 #combine numerical attributes with a newly converted attribute (categorical to numeric, Sex column)
22 df_X_new = pd.concat([df_X['Sex'], df_X_num], axis=1)
```

Gambar 0.2 Preprocessing Dataset

Buat model Decision Tree seperti pada Gambar 0.3 Decision Tree Model.



```
1 def dt(X, y):  
2     model = DecisionTreeClassifier(random_state=0)  
3     model = model.fit(X, y)
```

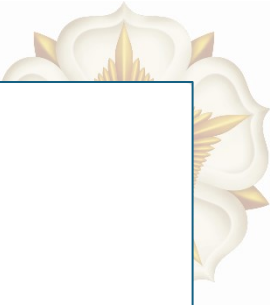
Gambar 0.3 Decision Tree Model

Untuk menyimpan hasil training model tersebut, pustaka joblib dapat digunakan. Lakukan import pustaka joblib dan simpan model seperti Gambar 0.4. Hasil model akan tersimpan pada file “dt_model.model” yang terdefiniskan di baris ke 7.

```
1 import joblib  
2  
3 def dt(X, y):  
4     model = DecisionTreeClassifier(random_state=0)  
5     model = model.fit(X, y)  
6     filename = 'dt_model.model'  
7     joblib.dump(model, filename)
```

Gambar 0.4 Menyimpan model hasil training

Gabungkan keseluruhan cuplikan kode tersebut menjadi satu sehingga script python akan membaca dataset, melakukan preprocessing data, training data, dan juga menyimpan hasil model. Kode utuh akan terlihat seperti Gambar 0.5.



```

1 import pandas as pd
2 from sklearn import preprocessing
3 from sklearn.tree import DecisionTreeClassifier
4 import joblib
5
6 def dt(X, y):
7     model = DecisionTreeClassifier(random_state=0)
8     model = model.fit(X, y)
9     filename = 'dt_model.model'
10    joblib.dump(model, filename)
11
12 if __name__ == '__main__':
13     #read the dataset
14     pd.options.display.max_columns=None
15     df_ori = pd.read_csv('hcvdat0.csv')
16     df_ori = df_ori.dropna()
17     #get the X and y
18     df_X = df_ori.drop(['Unnamed: 0', 'Category'], axis=1)
19     df_y = df_ori['Category']
20     #relabelling the class
21     df_y=df_y.replace('0=Blood Donor',0)
22     df_y=df_y.replace('0s=suspect Blood Donor',0)
23     df_y=df_y.replace('1=Hepatitis',1)
24     df_y=df_y.replace('2=Fibrosis',2)
25     df_y=df_y.replace('3=Cirrhosis',3)
26
27     #get the numerical attributes
28     df_X_num = df_X.drop(['Sex'], axis=1)
29     #convert single categorical column into numeric, label encoding
30     le = preprocessing.LabelEncoder()
31     df_X['Sex'] = le.fit_transform(df_X['Sex'])
32
33     #combine numerical attributes with a newly converted attribute (categorical to numeric, Sex column)
34     df_X_new = pd.concat([df_X['Sex'], df_X_num], axis=1)
35     #generate the model
36     print('Generate Decision Tree Model')
37     dt(df_X_new, df_y)

```

Gambar 0.5 Pembuatan Model DT Liver

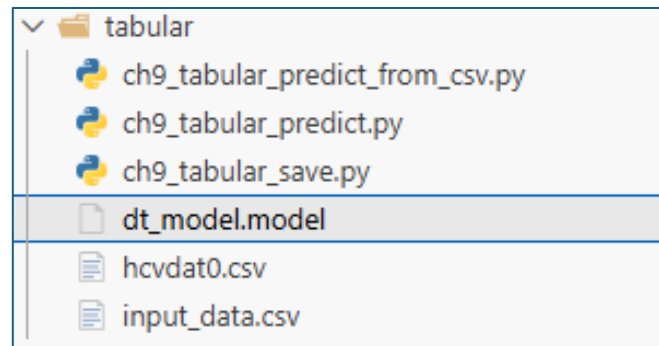
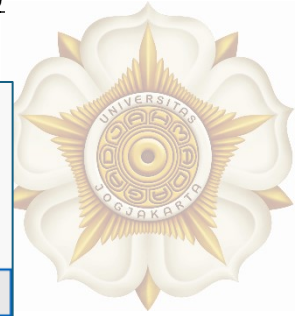
Jalankan script python pada Gambar 0.5. Jika berhasil maka akan terlihat seperti Gambar 0.6 dan akan menghasilkan file model seperti pada Gambar 0.7.

```

[13:54:05] (.venv) ~/test/flask/create_model/tabular
● $ python ./ch9_tabular_save.py
/home/ergeg/test/flask/create_model/tabular/./ch9_tabular_save.py:33: Future
sult.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.s
df_y=df_y.replace('3=Cirrhosis',3)
Generate Decision Tree Model

```

Gambar 0.6 Menjalankan script python



Gambar 0.7 File yang perlu disalin

3. Pembuatan Flask

Secara lengkap, pembuatan flask dapat dibaca pada dokumentasi resmi Flask.

<https://flask.palletsprojects.com/en/stable/quickstart/#a-minimal-application>

Flask memiliki *Web Server Gateway Interface* (WSGI) sehingga mempermudah dalam melakukan development. WSGI ini tentu berbeda dengan dedicated HTTP server. Untuk keamanan sebaiknya menggunakan dedicated HTTP server seperti NGINX, Apache, atau lainnya. Langkah-langkah yang perlu dilakukan jika ingin melakukan deployment secara nyata dapat dilihat pada dokumentasi berikut.

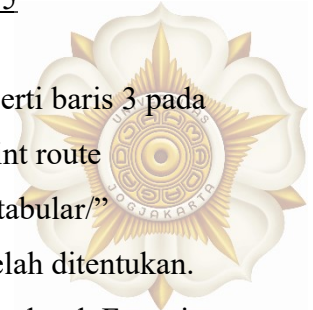
<https://flask.palletsprojects.com/en/stable/deploying/>

Dalam langkah-langkah berikut, akan digunakan WSGI yang telah disediakan oleh Flask. Perlu diingat bahwa WSGI ini mungkin tidak seaman dan tidak sebaik dedicated HTTP server.

Import pustaka-pustaka yang akan digunakan seperti Gambar 0.8.

```
1 import pandas as pd
2 import joblib
3 from typing import Optional
4 from flask import Flask, redirect, url_for, request, render_template
```

Gambar 0.8 Pustaka Deployment Flask



Untuk membuat aplikasi Flask Anda perlu membuat objek Flask seperti baris 3 pada Gambar 0.9. Objek flask ini dapat digunakan untuk membuat endpoint route menggunakan decorator route. Pada baris 5 Gambar 0.9, endpoint “/tabular/” didefinisikan. Endpoint ini akan mengembalikan sebuah aksi yang telah ditentukan. Dalam hal ini endpoint “tabular” akan mengembalikan halaman index.html. Fungsi `render_template` pada baris 7 Gambar 0.9 akan mencari file “index.html” yang berada di dalam folder “templates”. Baris 13 Gambar 0.9 akan digunakan untuk menjalankan Flask dengan WSGI. Kode tersebut bersifat opsional, Anda tetap dapat menjalankan script aplikasi ini dengan “`flask --app nama_file run`”.

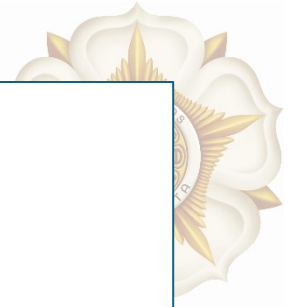
```
1  from flask import Flask, redirect, url_for, request, render_template
2
3  app = Flask(__name__)
4  #load index.html/ first page. receive input variable from user
5  @app.route("/tabular/")
6  def index():
7      return render_template('index.html')
8
9
10 if __name__ == "__main__":
11     #host= ip address, port = port number
12     #app.run(host='127.0.0.1', port='5001')
13     app.run()
```

Gambar 0.9 Minimal Flask Application

4. Integrasi Model

Melakukan integrasi model dilakukan pada aksi suatu endpoint. Anda dapat membuat endpoint tersebut mengirimkan JSON ataupun halaman html seperti yang akan dilakukan pada langkah-langkah berikut.

Tambahkan endpoint baru yang akan digunakan untuk menjalankan model AI. Endpoint ini akan menerima HTTP form, mengubah data input form tersebut ke bentuk input model, dan akan mengembalikan hasil prediksi serta memasukkan ke dalam result.html. Pastikan bahwa input telah sesuai dengan model yang dibuat. Endpoint form model Decision Tree klasifikasi penyakit hati dapat dilihat pada Gambar 0.10. Model AI di endpoint ini dipanggil pada baris 17-18.



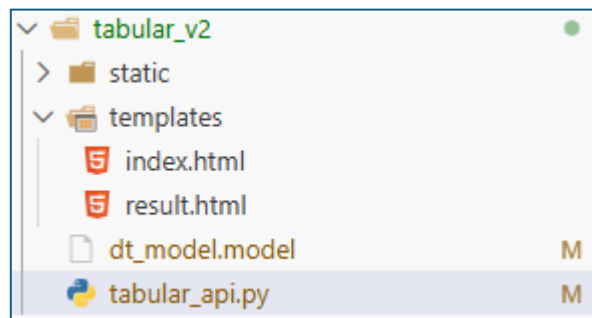
```

1 @app.route('/tabular/result/', methods=["POST"])
2 def prediction_result():
3     #receiving parameters sent by client
4     age = request.form.get('age')
5     sex = request.form.get('sex')
6     alb = request.form.get('alb')
7     alp = request.form.get('alp')
8     alt = request.form.get('alt')
9     ast = request.form.get('ast')
10    bil = request.form.get('bil')
11    che = request.form.get('che')
12    chol = request.form.get('chol')
13    crea = request.form.get('crea')
14    ggt = request.form.get('ggt')
15    prot = request.form.get('prot')
16    #load the trained model.
17    filename = 'dt_model.model'
18    loaded_model= joblib.load(filename)
19    # helper converters
20    def _to_float(x: Optional[str]) -> float:
21        return float(x) if x is not None and x != '' else float('nan')
22
23    def _encode_sex(x: Optional[str]) -> int:
24        if x is None:
25            return 0
26        s = str(x).strip().lower()
27        if s in ('m', 'male', '1', 'true'):
28            return 1
29        if s in ('f', 'female', '0', 'false'):
30            return 0
31        try:
32            return int(float(s))
33        except Exception:
34            return 0
35
36    sex_enc = _encode_sex(sex)
37
38    # create new dataframe matching training feature order
39    # Training used: df_X_new = pd.concat([df_X['Sex'], df_X[numb], axis=1)
40    columns_order = ['Sex', 'Age', 'ALB', 'ALP', 'ALT', 'AST', 'BIL', 'CHE', 'CHOL', 'CREA', 'GGT', 'PROT']
41    df_input = pd.DataFrame(columns=columns_order)
42    df_input.loc[0] = [
43        sex_enc,
44        _to_float(age),
45        _to_float(alb),
46        _to_float(alp),
47        _to_float(alt),
48        _to_float(ast),
49        _to_float(bil),
50        _to_float(che),
51        _to_float(chol),
52        _to_float(crea),
53        _to_float(ggt),
54        _to_float(prot),
55    ]
56    #make prediction
57    #print(df_input)
58    result = loaded_model.predict(df_input)
59    #print(result)
60    for i in result:
61        int_result = int(i)
62        if(int_result==0):
63            decision='Suspect Blood Donor'
64        elif (int_result==1):
65            decision='Hepatitis'
66        elif (int_result==2):
67            decision='Fibrosis'
68        elif (int_result==3):
69            decision='Cirrhosis'
70        else:
71            decision='Not defined'
72    #print('Disease is ', decision)
73    #return the output and load result.html
74    return render_template('result.html', age=age, sex=sex, alb=alb, alp=alp, alt=alt, ast=ast, bil=bil,
75                           che=che, chol=chol, crea=crea, ggt=ggt, prot=prot, status=decision)
76

```

Gambar 0.10 Endpoint Prediksi (tabular_api.py)

Jangan lupa untuk memindahkan model yang telah dibuat pada tahap nomor 2 ke direktori aplikasi Flask. Struktur lengkap proyek akan terlihat seperti pada Gambar 0.11 Struktur Proyek Flask. Perlu diperhatikan bahwa tiap versi pustaka dan python dapat menghasilkan hasil model yang berbeda, untuk itu environment pada aplikasi Flask dan pembuatan model dipastikan memiliki environment yang sama. Anda dapat menjalankan pembuatan model ulang jika terjadi error karena ketidakcocokan environment.



Gambar 0.11 Struktur Proyek Flask

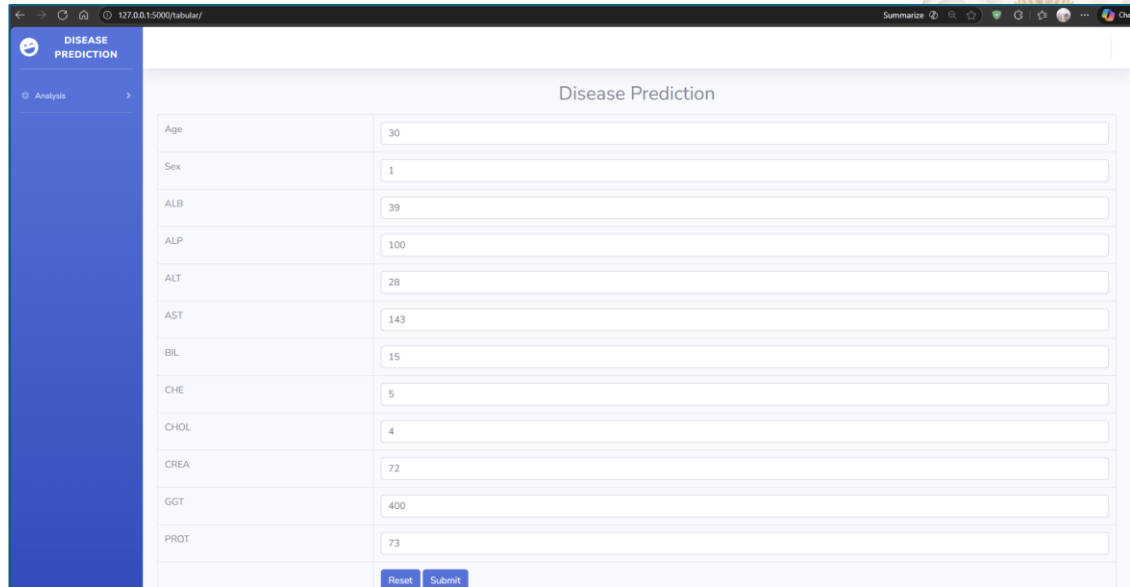
5. Menjalankan aplikasi Flask dengan WSGI

Jalankan script aplikasi Flask. Jika berhasil maka akan tertampil seperti cuplikan

```
[14:48:28] (.venv) ~/test/flask/web/tabular_v2
o $ python ./tabular_api.py
* Serving Flask app 'tabular_api'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Gambar 0.12 Menjalankan Flask

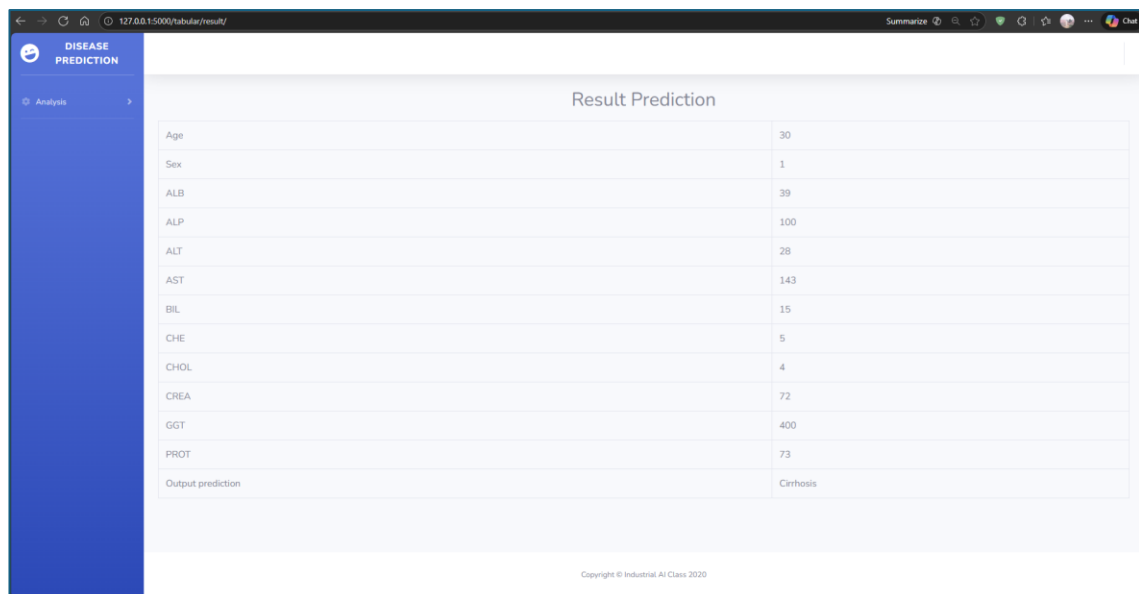
Buka url 127.0.0.1:5000/tabular, maka halaman Gambar 0.13 Endpoint Tabular akan tertampil. Masukkan input-input data kemudian dapat mencoba menekan aksi submit.



Disease Prediction	
Age	30
Sex	1
ALB	39
ALP	100
ALT	28
AST	143
BIL	15
CHE	5
CHOL	4
CREA	72
GGT	400
PROT	73
Reset Submit	

Gambar 0.13 Endpoint Tabular

Berikut hasil prediksi yang dihasilkan oleh model AI dan tampilan dibuat oleh Flask.



Result Prediction	
Age	30
Sex	1
ALB	39
ALP	100
ALT	28
AST	143
BIL	15
CHE	5
CHOL	4
CREA	72
GGT	400
PROT	73
Output prediction	Cirrhosis

Copyright © Industrial AI Class 2020

Gambar 0.14 Endpoint Result Tabular

10.5. TUGAS & ANALISIS

Silakan menggunakan dataset dari proyek kelompok untuk melakukan pelatihan model, kemudian buatlah dashboard dengan Flask. Setelah itu, buatlah laporan praktikumnya.

10.6. REFERENSI

<https://flask.palletsprojects.com/en/stable/installation/>

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.

